

CSS answer key

Alternative solutions are acceptable when they produce the required behaviour and use the CSS concepts requested in the exercise.

Section A: CSS setup, selectors, and basic styling

Exercise 1: Repair the stylesheet connection

The HTML refers to `style.css`, but the supplied file is named `styles.css`.

Corrected link:

```
<link
  rel="stylesheet"
  href="./styles.css"
>
```

Answers:

1. The incorrect reference is inside `index.html`.
2. The browser can still display the HTML because the HTML file loaded successfully.
3. The CSS does not appear because the path points to a file that does not exist.

The complete `<head>` should contain:

```
<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Reading room</title>

  <link
    rel="stylesheet"
    href="./styles.css"
  >
</head>
```

Exercise 2: Choose selectors that match the requirement

```
* {  
  margin: 0;  
}  
  
#page-header {  
  background-color: lightblue;  
}  
  
.intro {  
  color: darkblue;  
}  
  
.workshop-card {  
  border: 1px solid gray;  
}  
  
.workshop-list h2 {  
  font-size: 1.5rem;  
}  
  
.workshop-card > p {  
  color: #57534e;  
}
```

Selector explanations:

- `*` selects every element.
- `#page-header` selects the element with that ID.
- `.intro` selects elements with that class.
- `.workshop-card` selects every workshop card.
- `.workshop-list h2` selects every `<h2>` nested anywhere inside `workshop-list`.
- `.workshop-card > p` selects only paragraphs that are direct children of a workshop card.

Exercise 3: Predict which colour is applied

The message appears in `darkred`.

```
#registration-message {  
  color: darkred;  
}
```

Answers:

1. The final colour is `darkred`.

2. The ID selector has the highest specificity.
3. Moving the `p` rule below the ID rule would not change the result. The ID selector is more specific.
4. If the ID rule were removed, the class selector would apply and the text would be `darkorange`.

Specificity order in this example:

```
ID selector
Class selector
Element selector
```

Exercise 4: Style link interaction states

```
.course-nav a:link {
  color: darkblue;
}

.course-nav a:visited {
  color: purple;
}

.course-nav a:hover {
  text-decoration: underline;
}

.course-nav a:active {
  color: red;
}
```

Answers:

1. `:link` applies before a destination has been visited.
2. `:visited` applies after the browser records that the destination was visited.
3. `:hover` applies while the pointer is over the link.
4. `:active` applies while the link is being activated, such as while the mouse button is pressed.

The selector begins with `.course-nav a` so the styles apply only to links inside the course navigation.

Section B: Box model, sizing, display, and overflow

Exercise 5: Add space in the correct place

```
.notice {  
  padding: 20px;  
  margin-bottom: 16px;  
  border: 2px solid #78716c;  
}
```

Answers:

1. `padding` adds space inside the border.
2. `margin-bottom` adds space outside the border and separates the notices.
3. If both values were written as padding, the content would have more internal space, but the two notice boxes would still touch each other.

The box model order is:

```
Content  
Padding  
Border  
Margin
```

Exercise 6: Repair an incorrect display rule

Corrected CSS:

```
.resource-nav a {  
  display: block;  
  margin-bottom: 12px;  
}
```

Answers:

1. `position` was the incorrect property.
2. The correct declaration is `display: block`.
3. `display` is the property. `block` is one possible value for that property.
4. Each link begins on a new line and occupies a block-level box.

The original declaration was invalid because `block` is not a valid value for `position`.

Exercise 7: Control overflowing content

```
.announcement {  
  width: 280px;  
  height: 100px;  
  padding: 16px;  
  border: 2px solid #57534e;  
  overflow: auto;  
}
```

`overflow: auto` keeps the fixed height and adds scrolling only when the content does not fit.

`overflow: hidden` would remove the extra text from view. The complete announcement would no longer be available to the reader.

Section C: Positioning and stacking

Exercise 8: Predict static positioning

With `position: static`, the message remains in its normal document position.

```
.promotion {  
  position: static;  
  right: 40px;  
  bottom: 20px;  
  padding: 12px;  
  border: 2px solid orange;  
}
```

Answers:

1. The message appears where it normally occurs in the document flow.
2. `right: 40px` does not move it.
3. `bottom: 20px` does not move it.
4. Offset properties do not affect statically positioned elements.

To move the element relative to its original position:

```
.promotion {  
  position: relative;  
  right: 40px;  
  bottom: 20px;  
  padding: 12px;  
  border: 2px solid orange;  
}
```

With relative positioning:

- `right: 40px` moves the element 40 pixels to the left.
- `bottom: 20px` moves the element 20 pixels upward.
- Its original space remains reserved.

Exercise 9: Move an element without removing its original space

```
.course-badge {  
  position: relative;  
  left: 8px;  
  top: -6px;  
}
```

An equivalent version is:

```
.course-badge {  
  position: relative;  
  left: 8px;  
  bottom: 6px;  
}
```

Both versions move the badge:

- 8 pixels to the right
- 6 pixels upward

`position: relative` fits because the badge moves from its original position while its original space remains reserved.

`position: absolute` would remove the badge from normal flow and position it using an ancestor or the page as its reference.

Exercise 10: Position a label inside its card

Corrected CSS:

```
.session-card {  
  position: relative;  
  width: 320px;  
  padding: 24px;  
  border: 2px solid #57534e;  
}  
  
.live-label {
```

```
position: absolute;
top: 8px;
right: 8px;
}
```

Answers:

1. `.live-label` is absolutely positioned.
2. `.session-card` becomes the positioning reference.
3. `position: relative` is added to the parent because the label should be positioned inside the card rather than relative to the page.

The parent does not need offset values. Its positioned state is enough to establish the reference for the absolute child.

Exercise 11: Choose between fixed and sticky

Support button

```
.support-button {
  position: fixed;
  right: 20px;
  bottom: 20px;
}
```

`fixed` positions the button relative to the viewport. It remains in the same visible location while the document scrolls.

Section heading

```
.section-heading {
  position: sticky;
  top: 0;
  background-color: white;
}
```

`sticky` allows the heading to move normally with its section until it reaches the specified `top` position. It then remains near the top while its containing area continues to scroll.

A background colour is useful because page content may otherwise remain visible behind the sticky heading.

Exercise 12: Repair the stacking order

The banner appears above the dropdown because:

```
.banner {  
  z-index: 2;  
}
```

is higher than:

```
.dropdown {  
  z-index: 1;  
}
```

One correction is:

```
.dropdown {  
  position: absolute;  
  top: 30px;  
  left: 0;  
  width: 160px;  
  padding: 12px;  
  background-color: white;  
  border: 1px solid gray;  
  z-index: 3;  
}
```

Answers:

1. The banner has the higher stacking value.
2. Changing the dropdown to `z-index: 3` places it above the banner.
3. Padding changes the space inside the dropdown. It does not control which positioned element appears in front.
4. `display: block` changes layout behaviour. It does not correct the stacking order.

Section D: Responsive units and media queries

Exercise 13: Choose a responsive unit

1. Use part of the parent width

```
.content {  
  width: 90%;  
}
```

`%` is relative to the relevant size of the containing element.

2. Size text from the root font size

```
.page-heading {  
  font-size: 2rem;  
}
```

`rem` is relative to the root element's font size.

3. Use half of the viewport height

```
.hero {  
  min-height: 50vh;  
}
```

`vh` is relative to the viewport height.

4. Use one twentieth of the viewport width

```
.page-section {  
  padding-inline: 5vw;  
}
```

`5vw` is five percent of the viewport width, which is one twentieth of the viewport width.

5. Scale padding with the button font size

```
button {  
  padding: 0.5em 1em;  
}
```

`em` is relative to the element's current font size when it is used for padding.

Exercise 14: Repair a media query

Corrected CSS:

```
@media (max-width: 600px) {  
  .notice {  
    font-size: 1rem;  
  }  
}
```

Corrections:

1. Added `@` before `media`.
 2. Placed the condition inside parentheses.
 3. Replaced `=` with `:`.
 4. Added an opening and closing block for the media query.
 5. Replaced `font-size = 1rem` with `font-size: 1rem`.
-

Exercise 15: Predict breakpoint behaviour

Given:

```
.notice {  
  font-size: 1.25rem;  
  padding: 1.5rem;  
}  
  
@media (max-width: 600px) {  
  .notice {  
    font-size: 1rem;  
  }  
}
```

At `540px`

```
font-size: 1rem  
padding: 1.5rem
```

The media-query condition is true.

At `600px`

```
font-size: 1rem  
padding: 1.5rem
```

`max-width: 600px` includes exactly `600px`.

At `720px`

```
font-size: 1.25rem  
padding: 1.5rem
```

The media-query condition is false.

Answers:

1. The media query does not change the padding.
 2. The condition is true at exactly `600px`.
 3. The base `.notice` rule supplies the padding at every width.
-

Exercise 16: Rewrite the layout using a mobile-first approach

```
.course-nav a {  
  display: block;  
  margin-right: 0;  
  margin-bottom: 12px;  
}  
  
.course-title {  
  font-size: 2rem;  
}  
  
@media (min-width: 768px) {  
  .course-nav a {  
    display: inline;  
    margin-right: 16px;  
    margin-bottom: 0;  
  }  
  
  .course-title {  
    font-size: 3rem;  
  }  
}
```

The unprefixed rules form the narrow-screen base layout:

```
.course-nav a {  
  display: block;  
}  
  
.course-title {  
  font-size: 2rem;  
}
```

The `min-width` media query adds wider-screen changes from `768px` onward.

Final exercise: Build a responsive course information page

Completed `styles.css`

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background-color: #f5f5f4;
  color: #292524;
  font-family: Arial, sans-serif;
  line-height: 1.5;
}

.site-header,
.page-content,
.site-footer {
  width: 90%;
  max-width: 900px;
  margin-right: auto;
  margin-left: auto;
}

.site-header {
  padding-top: 1.5rem;
  padding-bottom: 1.5rem;
}

.site-name {
  color: #1e3a8a;
  font-weight: bold;
  text-decoration: none;
}

.course-nav {
  margin-top: 1rem;
}

.course-nav a {
  display: block;
  margin-bottom: 0.75rem;
  color: #1d4ed8;
  text-decoration: none;
}
```

```
.course-nav a:hover {
  color: #7c3aed;
  text-decoration: underline;
}

.course-hero {
  min-height: 50vh;
  padding: 2rem;
  background-color: #dbeafe;
}

.course-label {
  margin-top: 0;
  color: #1d4ed8;
  font-size: 0.875rem;
  font-weight: bold;
  text-transform: uppercase;
}

.course-title {
  margin-bottom: 1rem;
  font-size: 2rem;
}

.course-summary {
  max-width: 40rem;
}

.registration-link {
  display: block;
  margin-top: 1.5rem;
  color: #1d4ed8;
  font-weight: bold;
}

.course-notice {
  margin-top: 1.5rem;
  margin-bottom: 1.5rem;
  padding: 1rem;
  background-color: #fef3c7;
  border-left: 4px solid #f59e0b;
}

.schedule-section {
  margin-bottom: 2rem;
}

.session-card {
  position: relative;
  margin-bottom: 1rem;
  padding: 3.5rem 1.25rem 1.25rem;
```

```

    background-color: white;
    border: 1px solid #a8a29e;
}

.session-card h3 {
    margin-top: 0;
}

.session-status {
    position: absolute;
    top: 1rem;
    right: 1rem;
    padding: 0.25rem 0.5rem;
    background-color: #dcfce7;
    color: #166534;
    font-weight: bold;
}

.support-button {
    position: fixed;
    right: 20px;
    bottom: 20px;
    z-index: 10;
    padding: 0.75rem 1rem;
}

.site-footer {
    padding-top: 1.5rem;
    padding-bottom: 5rem;
}

@media (min-width: 768px) {
    .course-nav {
        margin-top: 1rem;
    }

    .course-nav a {
        display: inline;
        margin-right: 16px;
        margin-bottom: 0;
    }

    .course-hero {
        padding: 3rem;
    }

    .course-title {
        font-size: 3rem;
    }

    .registration-link {

```

```
        display: inline-block;
    }
}
```

Requirement review

Page foundation

```
body {
    margin: 0;
    background-color: #f5f5f4;
    color: #292524;
    font-family: Arial, sans-serif;
}
```

The body margin is removed, and the page receives its basic text and background styles.

```
.site-header,
.page-content,
.site-footer {
    width: 90%;
    max-width: 900px;
    margin-right: auto;
    margin-left: auto;
}
```

The three main areas:

- Use 90 percent of their available width
- Stop growing after 900 pixels
- Remain horizontally centred

Header and navigation

The narrow-screen navigation uses:

```
.course-nav a {
    display: block;
    margin-bottom: 0.75rem;
}
```

At 768px and wider:

```
.course-nav a {
    display: inline;
    margin-right: 16px;
}
```

```
margin-bottom: 0;  
}
```

The hover selector changes the link appearance only while the pointer is over a navigation link.

Hero section

The hero uses:

```
min-height: 50vh;  
padding: 2rem;
```

`vh` connects the minimum height to the viewport height.

`rem` connects the padding and text sizes to the root font size.

At wider viewports, the padding and title size increase.

Notice and session cards

The notice uses padding, a pale background, a left border, and vertical margins.

Each session card uses:

```
position: relative;
```

This makes it the positioning reference for its status label.

The larger top padding leaves space for the label so it does not cover the heading.

Support button

```
.support-button {  
  position: fixed;  
  right: 20px;  
  bottom: 20px;  
  z-index: 10;  
}
```

The button stays attached to the bottom-right corner of the viewport and appears above ordinary page content.

Responsive behaviour

At a narrow width:

- Navigation links appear on separate lines.
- The course title uses `2rem`.
- The hero uses `2rem` of padding.
- The registration link behaves as a block.

At `768px` and wider:

- Navigation links appear on one line.
- The title grows to `3rem`.
- The hero padding increases.
- The registration link participates in the text flow as an inline-level box.

`inline-block` is used for the registration link because it allows inline placement while preserving predictable margin and box-model behaviour.